

CO2: Scenario Based Assignment — Answers

Course: Artificial Intelligence (CSA17) | Assessment Tool 2

Q1. AI-Powered Drones for Emergency Medicine Delivery (Greedy Search vs A*)

i. Greedy Best-First Search behavior

Greedy Best-First Search selects the next node to expand purely based on the heuristic function $h(n)$ — here, straight-line distance to the goal — ignoring the actual cost $g(n)$ incurred so far. In this flood scenario, the drone would always move toward the path that looks closest to the destination in a straight line, without considering that a shorter-looking route might cross a flooded, high-cost, or blocked terrain segment.

- **Strengths:** Very fast, since it explores fewer nodes; low memory and computation overhead, which suits emergency situations needing quick decisions.
- **Weaknesses:** Not optimal and not complete in general graphs — it can get trapped in dead ends (e.g. a blocked road that looked promising) and can be misled entirely by an inaccurate heuristic, ignoring actual travel risk or cost. Under uncertainty, greedy search may repeatedly commit to routes that turn out worse or impassable.

ii. A* Search behavior

A* evaluates nodes using $f(n) = g(n) + h(n)$, combining the actual cost so far (g) with the estimated remaining cost (h). The drone balances how much effort/risk has already been spent against how promising the remaining path looks. As long as the heuristic is admissible (never overestimates true cost) and consistent, A* guarantees an optimal path. It would avoid a route that looks short (low h) but has high real terrain/weather cost (high g), favoring a route that is truly cheaper overall.

iii. Impact of heuristic accuracy

- **Greedy Search:** Heuristic accuracy is critical since it is the only factor driving expansion. An inaccurate or misleading heuristic (e.g. ignoring a flooded river crossing) can cause the drone to commit to a badly suboptimal or dead-end route.
- **A*:** A more accurate (but still admissible) heuristic makes A* more efficient by pruning unpromising branches faster, while still guaranteeing optimality. A weak or zero heuristic degrades A* toward Dijkstra's algorithm — still optimal, but slower. An inadmissible (overestimating) heuristic can break A*'s optimality guarantee.

iv. Impact of dynamic changes (blocked paths)

Both algorithms, as classical offline search methods, assume a static graph. If a path becomes blocked mid-execution, the drone must replan — either restart search from its current position or use an online/incremental variant (e.g. D* or Lifelong Planning A*) built for repeated replanning. Frequent replanning increases computational overhead and delay, which is risky in an urgent mission. Partial real-time updates mean the search must operate with incomplete information, so completeness and optimality guarantees weaken unless the algorithm continuously incorporates new data.

v. Recommendation

A* is the more suitable base algorithm because it guarantees optimal, safe routes when paired with a good admissible heuristic, and it does not get misled by dead ends the way Greedy does. However, since the environment is dynamic, a **replanning variant of A*** (such as D* Lite) should be used in practice, combining A*'s optimality with the ability to react quickly to newly blocked paths — balancing real-time responsiveness with route safety and near-optimal delay.

Q2. Smart City Traffic Signal Optimization (Local Search Methods)

Formulation as an optimization problem

This is naturally modeled as a local search / optimization problem rather than a path-finding problem: the state is a complete assignment of signal timings across all intersections, and the goal is to find the state that minimizes an objective function combining total waiting time, fuel consumption, and congestion. There is no single goal state to search toward — every candidate configuration has a quality score, and the aim is to find the best (or a very good) one. Traditional systematic search (BFS, DFS, A*) is inefficient here because the search space is combinatorially huge, a path to a goal is irrelevant (only the best state matters), and the environment changes continuously, making a static-problem search stale almost immediately.

i. Hill Climbing in this scenario

Hill Climbing starts from an initial timing configuration and repeatedly moves to a neighboring configuration with a better score (lower waiting time/congestion), stopping when no neighbor improves the current state.

- Easily gets stuck in local optima — a configuration better than all immediate neighbors but not the best overall.
- No mechanism to look ahead or backtrack.
- Sensitive to how neighboring states are defined (e.g. small timing adjustments only).

ii. Local maxima, plateaus, and ridges (real-world interpretation)

- **Local maxima:** A signal timing pattern that seems optimal for one intersection cluster but ignores a better city-wide arrangement — e.g. locally reducing wait time at one junction while worsening a nearby corridor's flow.
- **Plateaus:** Regions where many neighboring timing adjustments produce nearly identical waiting times, so the algorithm cannot tell which direction improves things.
- **Ridges:** Situations where the true improving direction requires simultaneously adjusting multiple intersections together (e.g. coordinating a green-wave corridor), but Hill Climbing only tries one adjustment at a time and each single step looks like it worsens performance.

iii. Simulated Annealing and Genetic Algorithms

- **Simulated Annealing (SA):** Allows occasional moves to worse states, with a temperature parameter controlling how often, decreasing over time. This lets the algorithm escape local maxima early on while converging toward a strong solution later.

- **Genetic Algorithms (GA):** Maintain a population of candidate timing configurations, combining (crossover) and mutating them across generations, and selecting the fittest. This explores multiple regions of the search space in parallel, making it less likely to get stuck in one local optimum and better able to discover coordinated multi-intersection solutions.

iv. Recommendation

Genetic Algorithms are best suited for this problem's scale and need for adaptability: they naturally handle large, multi-dimensional search spaces, can maintain diversity to avoid premature convergence, and can be re-run or continuously evolved as traffic patterns shift. Simulated Annealing is a reasonable lighter-weight alternative for smaller-scale tuning, but GA scales better to hundreds of intersections and adapts more robustly to continuously changing conditions.

Q3. Mars Rover Autonomous Exploration (Online Search)

i. Why an online search agent is required

Offline search assumes the agent has complete knowledge of the environment before acting and computes an entire solution path in advance. The rover cannot do this because the terrain is unknown and unmapped in advance — hazards like craters and rocks are discovered only as the rover senses its surroundings. An online search agent interleaves computation and action: it senses, decides the next action, executes it, then senses again — updating its plan as new information arrives, which is essential when a complete a-priori map does not exist.

ii. Challenges of partial observability and dynamic environments

- **Partial observability:** The rover only knows about terrain within its sensor range, so it may commit to a direction that later reveals an impassable hazard, forcing costly backtracking.
- **Dynamic environments:** Even previously explored terrain can change (dust settling, small rockslides), meaning the rover's internal model can become outdated and must be continually refreshed.
- Together these make guarantees of completeness and optimality much harder — the rover may need to explore more of the environment than a fully-informed agent would, sometimes retracing steps.

iii. Building and updating the internal model

As it moves, the rover incrementally constructs an internal map (often grid or graph-based) from sensor readings (cameras, LIDAR), marking cells as free, occupied, or unknown. When new sensor data contradicts or extends this model, the rover updates the corresponding map cells and revises its plan — a form of incremental/online mapping, similar in spirit to SLAM (Simultaneous Localization and Mapping). This model is then used to select next actions, favoring unexplored or promising regions while avoiding known hazards.

iv. Comparison with uninformed and informed offline search

Aspect	Uninformed Search	Informed Search (offline)	Online Search
Map knowledge	Full, no heuristic	Full, with heuristic	Unknown/partial, discovered as it goes
Planning	Full plan before acting	Full plan before acting	Plan-act-sense loop

Adapts to new info	No	No	Yes
Suitability here	Poor (needs full map)	Poor (needs full map + heuristic)	Necessary

v. Justification of best approach

An online search approach (e.g. online variants of A* or reactive/agent-based exploration strategies) is the most suitable, since it can adapt to the rover's evolving knowledge, respond immediately to newly discovered hazards, and does not require pre-existing map data that simply is not available on Mars. This directly matches the requirements of uncertainty-handling, adaptability, and safety in the scenario.

Q4. University Exam Timetabling (CSP)

Formulation as a CSP

- **Variables:** Each exam (one variable per course/subject to be scheduled), e.g. Exam_CS101, Exam_MATH201, etc.
- **Domains:** For each exam variable, the domain is the set of possible (time slot, exam hall) pairs it could be assigned to.
- **Constraints:** No student can have two exams in the same slot (any two exams sharing a student must be in different slots); the number of exams scheduled simultaneously cannot exceed the number of available halls; certain subjects must be scheduled before others (precedence constraints); an exam requiring a specific invigilator/faculty member cannot be scheduled when that person is unavailable; certain students need specific slot/hall arrangements (e.g. extra time, ground-floor access), restricting valid domain values for exams they are part of.

i. Backtracking search in this scenario

Backtracking search assigns exams to time slots/halls one at a time, checking constraints after each assignment. If an assignment violates a constraint (e.g. two exams for the same student end up in the same slot), the algorithm backtracks and tries a different value. This continues, depth-first, until a complete valid assignment is found or all options are exhausted.

ii. Constraint propagation (e.g. forward checking)

Forward checking looks ahead after each assignment: whenever an exam is assigned a slot/hall, it immediately removes now-inconsistent values from the domains of related, not-yet-assigned exams — e.g. if Exam_CS101 is fixed to Monday 9 AM, any other exam sharing a student with CS101 has "Monday 9 AM" removed from its domain. This detects dead ends much earlier, reduces the effective branching factor at each step, and can be extended further with arc consistency (e.g. AC-3) to propagate constraints across the whole constraint graph, catching more conflicts upfront.

iii & iv. Real-world challenges and best strategy for scalability

Challenges include the combinatorial explosion of variables/domains as courses and students grow, tightly interacting constraints (student overlaps ripple across many exams), and near-real-time re-scheduling needs when late changes occur (room unavailability, student requests). For scalability, the best strategy combines:

- Backtracking + forward checking/arc consistency to prune the search space early.
- Heuristics for variable and value ordering — e.g. Minimum Remaining Values (choose the most constrained exam next) and Least Constraining Value (pick the hall/slot that rules out the fewest future options) — which significantly improve performance on large CSPs.
- Optionally, local search/metaheuristics (e.g. min-conflicts) for very large instances where a full backtracking search becomes impractical, allowing a good-enough timetable to be found quickly and then refined.

Q5. Real-Time Strategy Game AI (Minimax & Alpha-Beta Pruning)

i. How Minimax models this problem

Minimax treats the game as a two-player, zero-sum adversarial search: it builds a game tree where the AI (maximizer) tries to choose moves maximizing its evaluated outcome, while the opponent (minimizer) is assumed to choose moves minimizing that same outcome. The algorithm recursively evaluates each possible move sequence down to some depth, propagating values back up the tree (max at the AI's turns, min at the opponent's turns) to choose the current best move — assuming rational, worst-case opposition.

ii. Computational challenges of large game trees

- The tree's size grows exponentially with depth and branching factor (number of legal moves per state), making full-depth search infeasible for a real-time strategy game with many possible actions.
- Real-time constraints mean the AI has only a limited time budget per decision, so it cannot explore anywhere near the full tree.
- Evaluating complex game states (many units, resources, positions) makes each node's evaluation function itself expensive to compute, compounding the cost of deep search.

iii. Alpha-Beta pruning

Alpha-Beta pruning improves Minimax by tracking two bounds — alpha (best value the maximizer can guarantee so far) and beta (best value the minimizer can guarantee so far) — while traversing the tree. If, at any node, the current branch is found to be unable to influence the final decision (i.e. $\alpha \geq \beta$), that branch is pruned — the AI stops exploring it entirely, since a rational opponent would never allow that outcome to occur. This does not change the final decision (same result as full Minimax) but can, with good move ordering, reduce the effective branching factor dramatically — in the best case, roughly halving the exponent, allowing much deeper search within the same time budget.

iv. Designing an evaluation function

Since the tree cannot be searched to completion, non-terminal states must be scored by a heuristic evaluation function combining factors such as:

- Material/unit balance (own vs. enemy unit count and strength).
- Positional advantage (map control, resource proximity, defensible terrain).
- Resource economy (gathered resources, production capacity).
- Threat/vulnerability level (exposure to attack, unit health).

- Tempo/momentum (who is currently on the offensive).

These factors should be weighted and combined into a single numeric score, tuned (manually or via learning) to reflect true strategic value at that depth.

v. Recommended strategy for balancing optimality and real-time performance

Use Alpha-Beta pruning with iterative deepening (search to depth 1, then 2, then 3, etc., keeping the best result found before the time budget expires) combined with good move ordering (trying likely-best moves first to maximize pruning) and a well-tuned heuristic evaluation function for cutting off search at a feasible depth. This sacrifices guaranteed global optimality (impossible under real-time constraints) for a strong, good-enough move chosen reliably within the time limit — the standard practical approach used in real-time game AI.